AHLT Lab Project - Final Report

Task 2: Drug–Drug Interaction (DDI) Extraction

Feature-based ML vs. Neural Networks vs. Large Language Models on SemEval-2013 Task 9

Table of Contents

Lab Group 10 - authors and primary responsibility:

Member	Primary focus across the three systems
Ralph Khairallah (Exchange)	Feature engineering, fine-tuning campaigns, error analysis
Francesco Dorati (Exchange)	Neural architecture sweeps, evaluation, figures

Repository: `github.com/francesco-dorati/AHLT-project`
Shared task: `SemEval-2013 Task 9 (DDIExtraction)`
Date: May 20, 2026

---

1 Executive Summary

At a glance

This report covers **Task 2 of the AHLT lab project: Drug–Drug Interaction (DDI) extraction**, framed as a sentence-level classification problem on the SemEval-2013 Task 9 corpus. We build, tune and compare **three systems** on the *same* data and the *same* evaluator: a feature-based **Machine-Learning** classifier (2.1), a **Neural-Network** BiLSTM+CNN (2.2), and a **Large Language Model** fine-tuned with LoRA (2.3). The headline result is the *mirror image* of Task 1 (NERC): on this relational task the classical ML system **wins** (test macro-F1 = 66.8 %), the neural network is within 1.2 pp (65.6 %), and the fine-tuned 3B LLM trails both by ≈27 pp (39.8 %). The dominant difficulty throughout is the **85 % `null` class imbalance** and the fact that the signal lives in *syntax* (dependency paths between the two target drugs) — information the ML model encodes directly but the LLM has to reconstruct.

Headline fact	**Value**
Task framing	sentence-level 5-class problem (advise / effect / int / mechanism / `null`)
Class imbalance	≈85 % of all pairs are `null` (no interaction)
Best system (test macro-F1)	**2.1 ML** two-stage on `mod_best2` = **66.8 %**
Neural network (test)	2.2 BiLSTM + relative-position embedding = 65.6 %
Fine-tuned LLM (test)	2.3 Llama-3.2-3B, LoRA r=32, `prompts02` = 39.8 %
Rule-based floor (test)	2.0 cue-word baseline = 26.9 %
Biggest lever, ML	a two-feature combo + a two-stage classifier (+2.5 pp over reference)
Biggest lever, NN	input representation (prefix + entity-type marker, +10 pp)
Biggest lever, LLM	LoRA rank *and* prompt quality (∼+5 pp each)
Headline cross-task lesson	on a *relational* task, classical NLP beats a small LLM

Table 1: Headline numbers for the three delivered DDI systems. All test-set figures use the shared `util/evaluator.py DDI` script and report macro-F1 (M.avg) over the four positive classes.

2 Introduction

2.1 Context

Drug–drug interactions are a leading cause of avoidable adverse clinical events, and most of the evidence for them is locked in unstructured biomedical text: drug labels, case reports, and scientific abstracts. Task 1 of this project located and typed the drug *mentions* (NERC). Task 2 takes the next step: given a sentence and two drug entities already marked in it, decide whether the sentence *states* that the two drugs interact, and if so, which *type* of interaction it is. Crucially — as the lab brief stresses — we are not deciding whether two drugs interact in reality, but whether the *target sentence says so*. This makes DDI a **sentence-level relation-classification** problem rather than a sequence-labelling one.

The task is intrinsically harder than NERC for two reasons. First, the label distribution is brutally skewed: roughly **85 % of all candidate pairs are `null`** (the sentence asserts no interaction for that pair), so a model that simply predicts "no interaction" everywhere already scores 85 % accuracy while being useless. Second, the discriminative signal is *relational and syntactic*: it lives in the dependency path between the two target drugs and in the trigger verbs along that path ("*X potentiates Y*", "avoid co-administration of *X with Y*"), not in the surface form of any single token.

2.2 Objectives

We deliver and compare three DDI systems spanning the main paradigms in modern NLP, exactly as the lab specification requires:

- **System 2.1 — feature-based Machine Learning**: a Maximum-Entropy (logistic-regression) classifier over hand-crafted lexical and *syntactic* features (dependency paths, clue verbs, entity-type pairs), extended with a two-stage null-vs-positive router.
- **System 2.2 — Neural Networks**: the shipped BiLSTM+CNN sentence classifier, improved through input-representation engineering (prefix / suffix / entity-type / relative-position channels), architecture variants and a multi-seed robustness audit.
- **System 2.3 — Large Language Models**: a generative single-label classifier, evaluated both few-shot (no training) and after LoRA fine-tuning of 4-bit-quantised Llama-3.2-3B and Qwen-2.5-3B, with prompt-design and fine-tuning-recipe ablations.

For each system we document the experiments performed, the effect of each tried variation, and the best obtained results, following the deliverable structure of the brief. The closing section (7) gives the required cross-system comparison along the four axes the brief names: *development effort*, *computational cost*, *explainability and controllability*, and *resulting performance*.

3 Data, Evaluation, and Reproducibility

3.1 Dataset

The data is the DDI Extraction 2013 corpus (SemEval-2013 Task 9), combining **DrugBank** drug-label text and **MedLine** research abstracts. Sentences come with gold drug entities; every *ordered pair* of entities within a sentence is a classification instance whose gold label is one of four interaction types or `null`:

- **advise**: a recommendation or warning about combining the drugs ("*should be avoided*").
- **effect**: a clinical effect / pharmacodynamic outcome of the combination.
- **mechanism**: a pharmacokinetic mechanism (absorption, metabolism, clearance).
- **int**: a generic, untyped interaction mention ("*X interacts with Y*").
- **null**: the sentence does *not* assert an interaction for this pair.

Split	Sent.	Pairs	advise	effect	int	mech.	null
Train	5 343	22 793	695	1 439	201	1 016	19 442
Devel	1 477	4 877	143	316	43	261	4 114
Test	1 455	5 838	209	293	40	344	4 952
% null			train 85.3 %		devel 84.4 %		test 84.8 %

Table 2: DDI-2013 corpus statistics. The `int` class is extremely rare (40–201 pairs) and `null` dominates every split, which is the central modelling challenge of the task.

3.2 Evaluation protocol

The provided `util/evaluator.py DDI` scores predictions at the *pair* level. A prediction is correct only if both the "interacts" decision and the interaction *type* match the gold label. The evaluator reports precision, recall and F1 *per positive class*, plus two aggregates: **macro-F1 (M.avg)**, averaging the four positive-class F1s equally, and **micro-F1 (m.avg)**, pooling all positive predictions. `null` pairs are excluded from F1 by construction (predicting `null` can only cost recall on the positives, never earn credit). Because the four positive classes are wildly different in frequency, **macro-F1 is the primary metric** throughout this report — it is the number a report must defend, since micro-F1 is dominated by the two common classes (`effect`, `mechanism`).

3.3 Reproducibility

Each system lives in its own directory (`code/2.1.DDI-ML/`, `2.2.DDI-NN/`, `2.3.DDI-LLM/`) with a full experiment log in `NOTES.md`; every code change is tagged `[MOD-2.x]` in source. Systems 2.1 runs entirely on CPU; 2.2 and 2.3 run on the `Boada` GPU cluster via `sbatch`. All figures in this report are regenerated from the logged `.stats` files by the `report/generate_plots_2_*.py` scripts, so every number is traceable to a committed result file.

4 System 2.1 - DDI with Machine Learning

4.1 Baseline and reference classifier

The rule-based starting point (2.0) classifies a pair by matching ∼50 hand-curated cue words found between the two drugs. It reaches only **test M-F1 = 26.9 %**: it nails `int` (its cue list contains "interact") but collapses on `advise` (0 % on devel), because its cue list there is over-specific drug names. This is our floor.

The provided ML reference replaces cue-matching with a classifier over a rich shipped feature set — entity types, the `same-drug` flag, the lowest-common-subsumer (LCS) lemma/PoS, **nine dependency-path variants**, words/lemmas in the path, and four pattern families (verb-LCS, verb-function, words-in-between, words-outside). We compared the two classifiers the brief suggests:

Classifier	advise	effect	int	mech.	M-F1	m-F1
MEM (LogisticRegression)	62.6	65.4	69.2	60.0	**64.3**	63.2
SVM (rbf, default)	49.2	60.2	61.5	49.0	55.0	54.3

Table 3: Reference classifiers on devel (per-class F1). MEM beats SVM on *every* class; the SVM is heavily precision-biased (recall <45 % everywhere). MEM (a maximum-entropy / logistic-regression model) is therefore the carrier for all feature engineering.

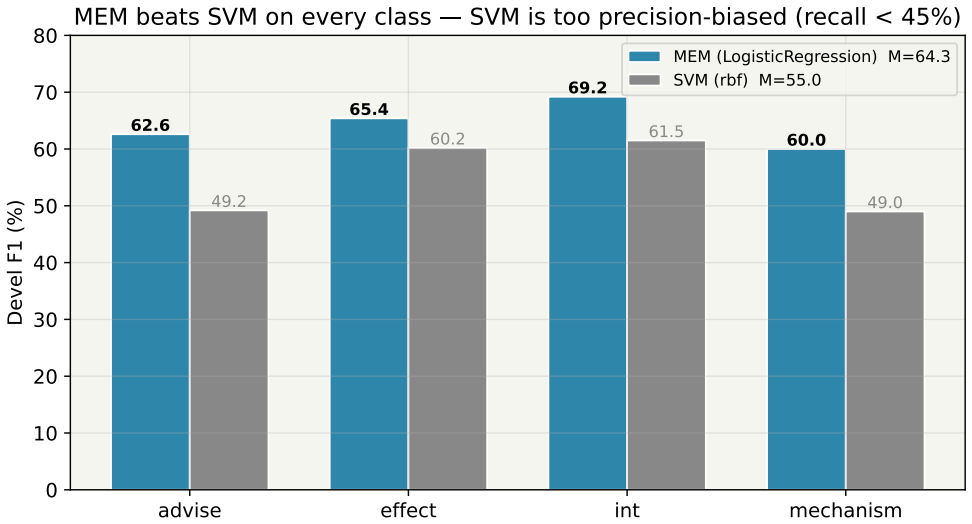


Figure 1: MEM vs. SVM per-class F1 on devel. The high-dimensional, sparse feature space is exactly logistic regression's regime; the rbf-SVM under-fits the positive classes.

4.2 Feature-engineering experiments

Following the brief's instruction to pay *special attention to syntax-based features*, we added feature mods one at a time on top of the reference set and kept only those that helped on devel.

Mod	What it adds	ΔM
mod1	distance + sentence-position / length buckets	−1.9
mod2	pharmacology clue-verbs (lemma + position vs. the pair)	−0.1
mod3	third-entity context (other entities in sentence / in path)	−0.3
mod4	combined entity-type pair feature (`typeE1_typeE2`)	−0.2
mod5	negation cues near the LCS / in the path	−1.3
mod_best2	**mod2 + mod3 together**	**+1.1**

Table 4: Additive feature mods, Δ macro-F1 vs. the reference (64.3). Every single mod is neutral-to-negative in isolation, but the `mod2+mod3` *combination* (clue-verbs + third-entity context) synergises to +1.1 pp. Adding `mod4` on top helps on test but hurts devel, so devel-selection keeps it out.

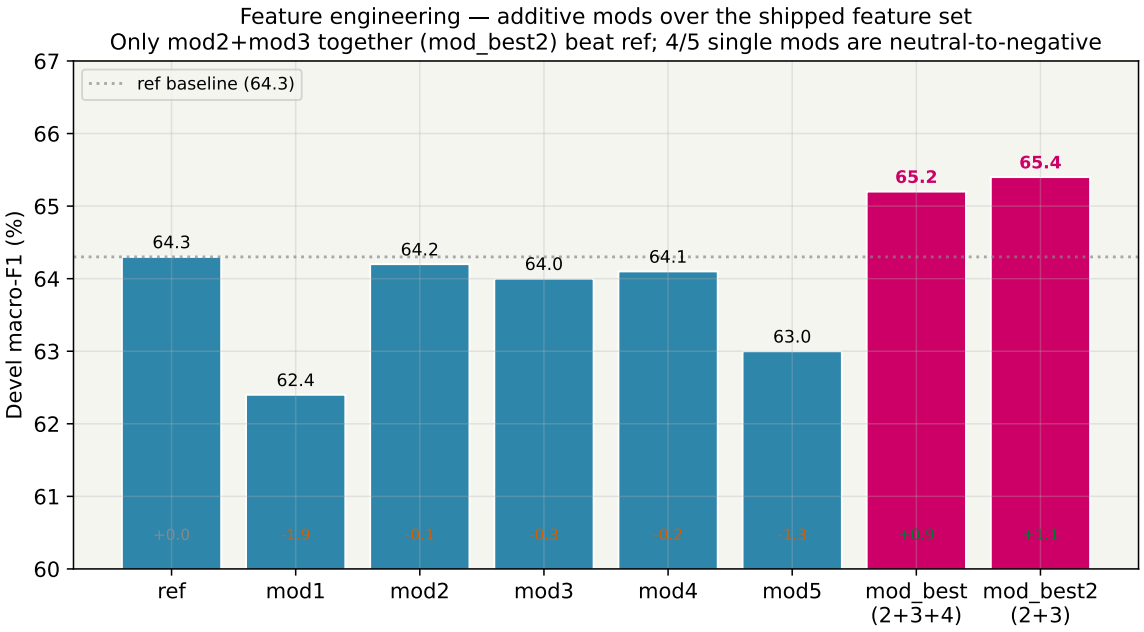


Figure 2: Feature-mod sweep on devel macro-F1. The lesson mirrors Task 1: the shipped feature set is already well-tuned, so isolated additions dilute a sparse model, but a carefully chosen *combination* (`mod_best2`) wins.

Key observation

The single most important finding of the feature campaign is that **combinations beat individuals**: `mod2` and `mod3` are each ≈neutral alone, yet together they lift macro-F1 by +1.1 pp. The informative clue-verb and third-entity signals cooperate in a way the linear model cannot reconstruct from either one alone. A full hyperparameter sweep (solver ∈ {lbfgs, saga}, $C \in [0.1, 5]$, penalties l1/l2/elasticnet) added *at most* +0.1 pp on top — **feature engineering, not hyperparameter tuning, is where the gains are**.

4.3 The two-stage classifier - the largest single jump

The reference is a flat 5-way classifier fighting both the null-vs-positive imbalance *and* the inter-positive boundary at once. We decomposed it: **stage 1** is a binary null-vs-positive router (logistic regression); **stage 2** is a 4-way classifier trained on positive-only data. A pair is routed to stage 2 only when stage 1's $P(\text{positive})$ exceeds a threshold tuned on devel.

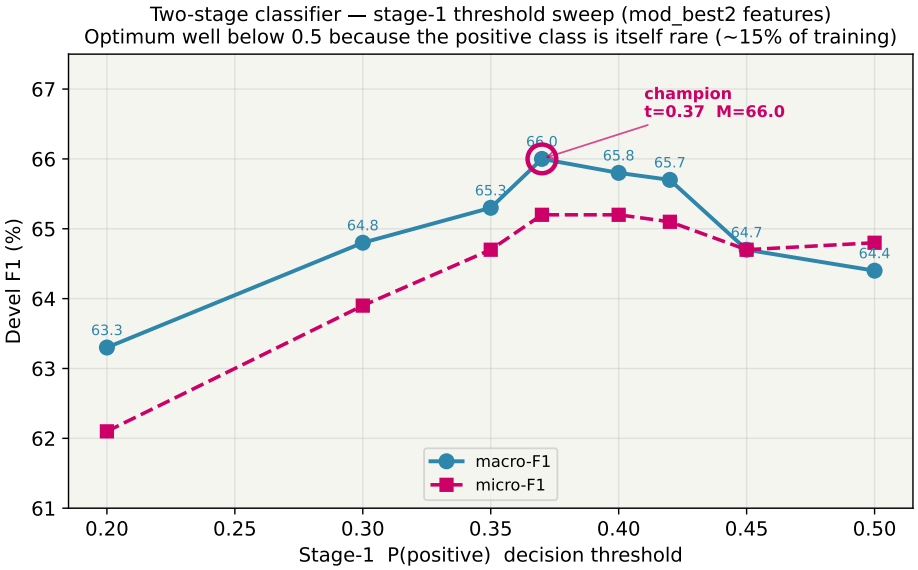
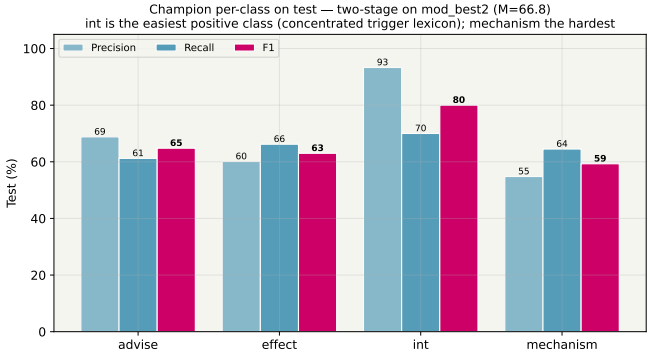


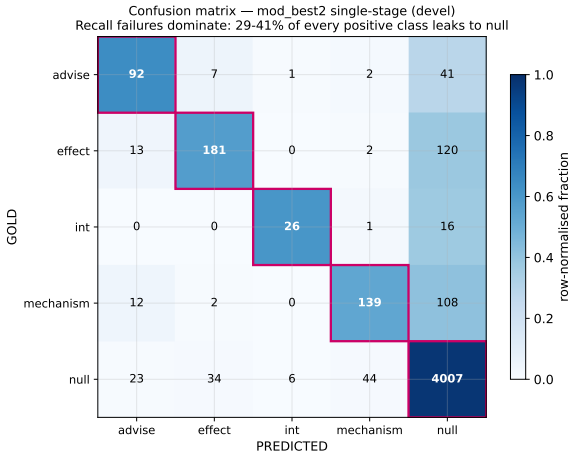
Figure 3: Stage-1 threshold sweep on devel. The optimum (**t = 0.37**, M-F1 = 66.0) sits well below 0.5 because the "positive" class is itself rare (≈15 % of training pairs): demanding 0.5 probability would reject too many borderline true positives.

The two-stage router at t = 0.37 is the **champion of System 2.1**: devel M-F1 = 65.9, **test M-F1 = 66.8**, a +2.5 pp devel and +0.9 pp test gain over the single-stage `mod_best2`. It works by trading precision for recall on every class — exactly what we want once stage 1, not the 4-way head, owns the imbalance decision.

4.4 Best results, per-class and error analysis



(a) Champion per-class P/R/F1 on test.



(b) Confusion matrix (`mod_best2`, devel).

Figure 4: System 2.1 champion analysis. `int` is the easiest positive class (test F1 = 80.0, a concentrated trigger lexicon); `mechanism` is the hardest (diffuse phrasing). The confusion matrix shows the dominant error is *recall* failure — 29–41 % of every positive class leaks to `null` — not inter-positive confusion, which is small.

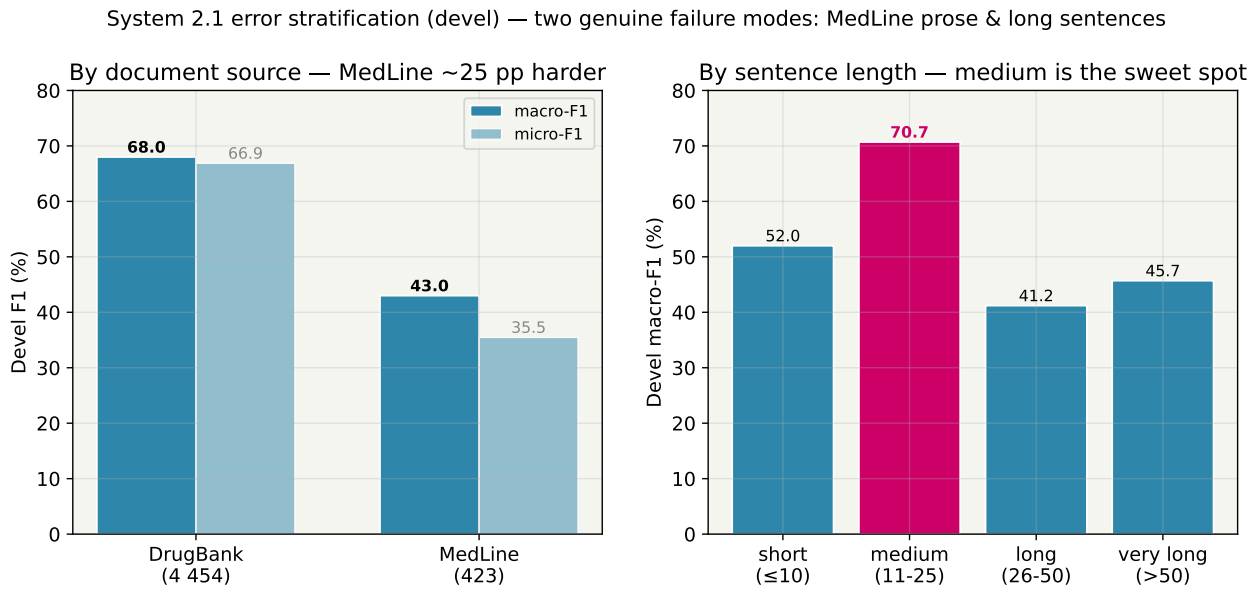


Figure 5: Error stratification on devel. **DrugBank pairs are ≈25 pp easier than MedLine** (standardised drug-label phrasing fits the clue-verb features; research-abstract prose does not), and **medium-length sentences are the sweet spot** — both very short (too few words for the verb features to fire) and long (>25 words, noisy dependency paths) sentences are harder.

Deep dive — five extensions that all fall short

After the headline, we tried five further refinements: stage-2 class-balancing, train+devel retraining, per-class thresholds, a mechanism-specific trigger lexicon (`mod8`), and single/two-stage ensembling. *Per-class thresholds* gave a spectacular +2.5 pp on **devel** but −1.5 pp on **test** — the four free thresholds overfit the ∼700 devel positives. Every extension showed the same pattern: devel gains that do not survive test. At this data scale, the single global threshold acts as an implicit regulariser, and the plain two-stage classifier is the honest, devel-selected champion.

4.5 Discussion

Inspecting the learned weights confirms the classifier is well-behaved: the strongest single feature in the whole model is `same-drug` (a drug mentioned twice is almost never a real interaction), and each class's top features are linguistically sensible (modal verbs "*should*" for `advise`; pharmacokinetic vocabulary "*absorption*", "*metabolism*" for `mechanism`). The two genuine failure modes are MedLine-style research prose and long sentences with diffuse syntax — both out-of-domain for the shipped syntactic feature set. **System 2.1 final: two-stage on `mod_best2`, lbfgs, t = 0.37 → test M-F1 = 66.8** (+39.9 pp over the rule-based 2.0).

5 System 2.2 - DDI with Neural Networks

5.1 Starting point

The shipped network (despite being called "CNN" in the brief) is a **BiLSTM-then-CNN hybrid**: three embedded input channels (lower-cased word, lemma, PoS) are concatenated, run through a BiLSTM(250→200), then a 1-D convolution and a global max-pool, and a linear layer projects to the 5 classes. Crucially, the network sees *only* word/lemma/PoS sequences — it has **none** of the dependency-path features that carry most of the DDI signal in 2.1. The reference network reaches devel M-F1 = 55.8, fully **10 pp below the ML reference**, and this gap defines the opportunity.

5.2 Input-representation experiments

The brief asks us to experiment with input encodings; this turned out to be by far the largest lever.

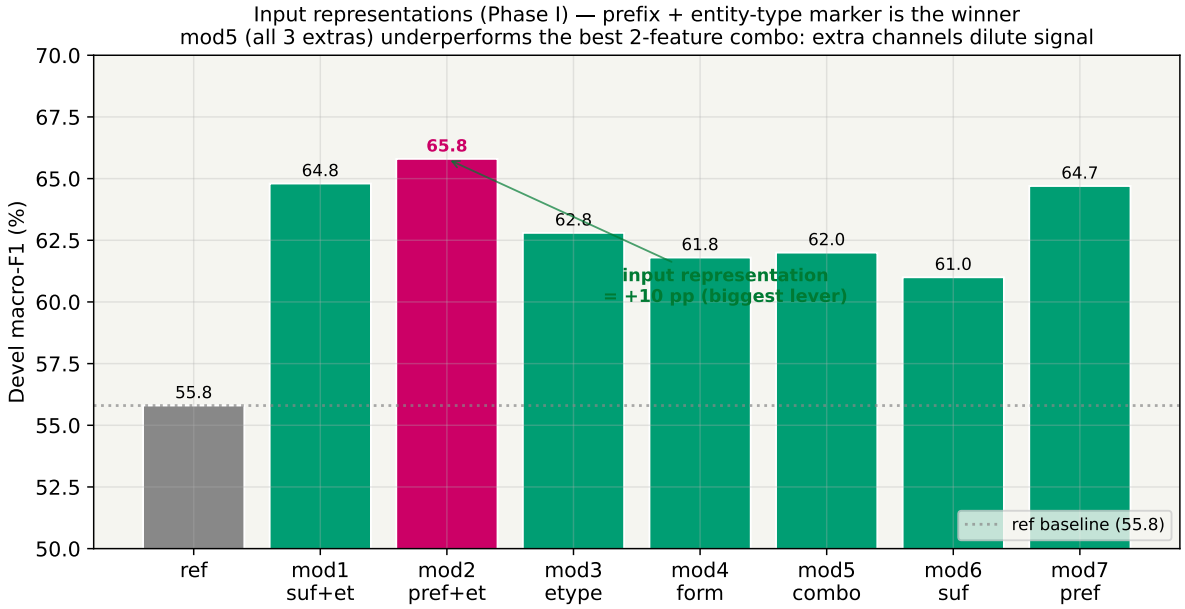
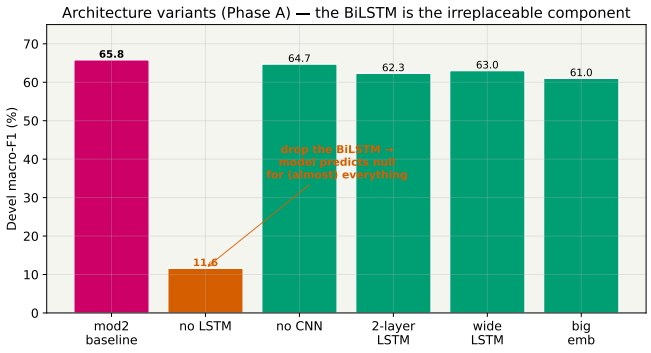
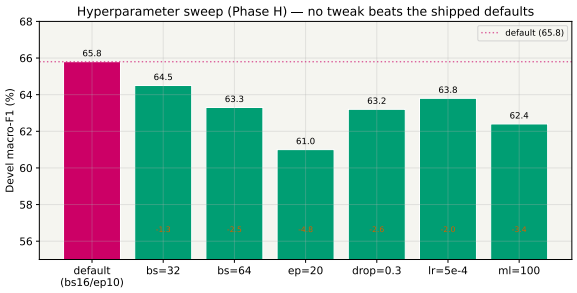


Figure 6: Phase I input-representation sweep (devel macro-F1). Adding a **prefix-3 channel plus an entity-type marker** (`mod2`) lifts macro-F1 by **+10 pp** over the shipped baseline — the single biggest improvement anywhere in System 2.2. Combining all three extra channels (`mod5`) does *worse* than the best two-channel combo: at this data scale extra input channels dilute the signal.

5.3 Architecture and hyperparameter experiments



(a) Architecture variants.



(b) Hyperparameter sweep.

Figure 7: Phase A/H. **Removing the BiLSTM collapses the model to M-F1 = 11.6** (it predicts `null` for almost everything) — the BiLSTM is the irreplaceable component, while the CNN contributes only ≈1 pp. No depth/width change and no hyperparameter tweak beats the shipped defaults; deeper/wider variants overfit at this data scale.

5.4 Relative-position embedding - the champion modification

The deep dive's most impactful idea was a **relative-position embedding** (`mod9`): for every token, embed its bucketed distance to `<DRUG1>` and to `<DRUG2>`. This gives the network the explicit "near the pair vs. far away" signal that a plain BiLSTM lacks — a standard relation-extraction trick.

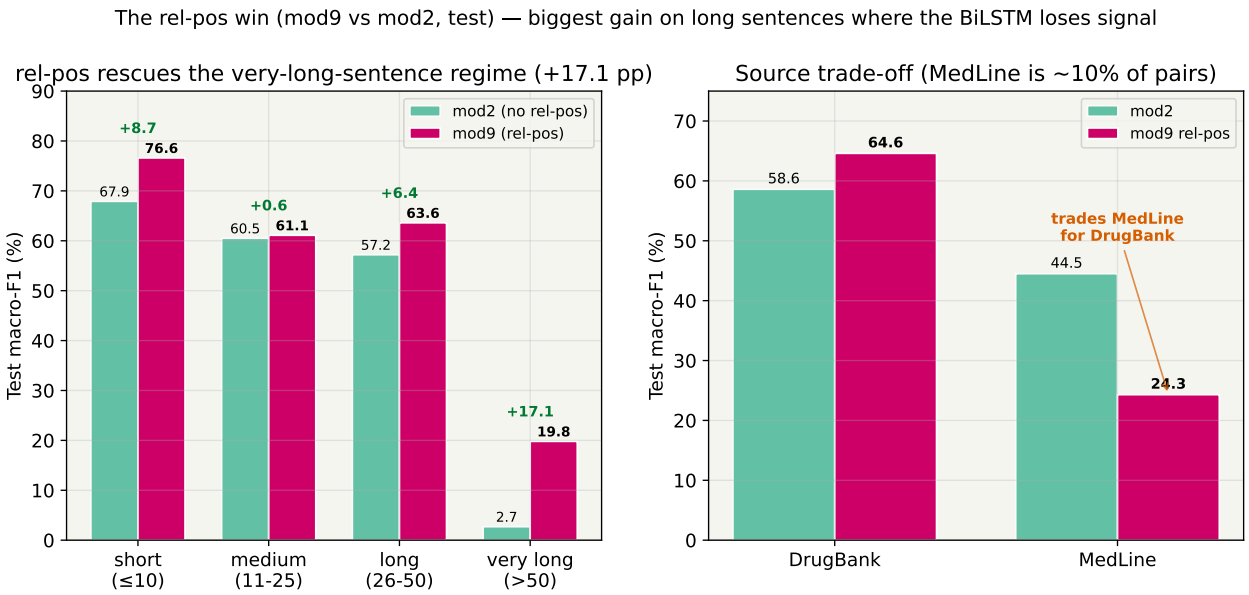
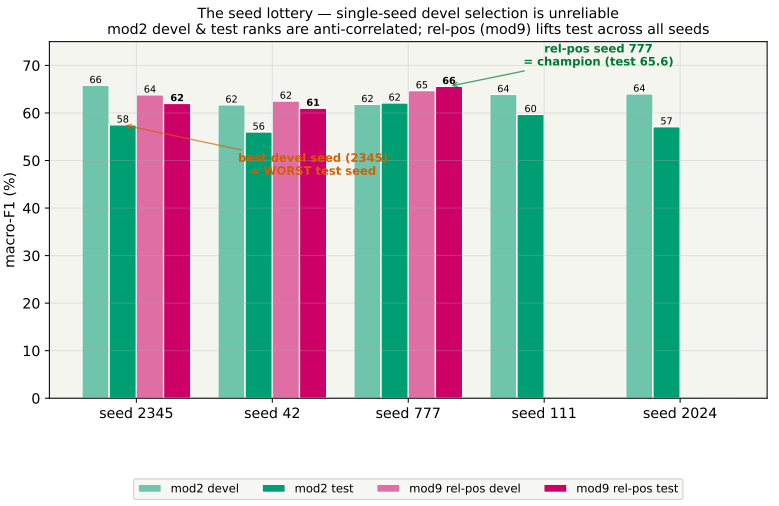


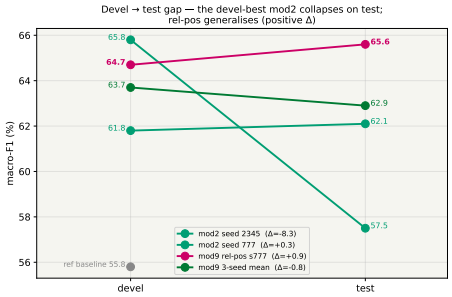
Figure 8: Relative-position (`mod9`) vs. `mod2` on test. Rel-pos rescues exactly the regime the BiLSTM struggles with: **+17.1 pp on very-long sentences** (from a near-dead 2.7 to 19.8) and +6.4 pp on long ones. It trades away some MedLine accuracy (a small ≈10 % subset) for a large DrugBank and long-sentence gain.

5.5 The seed lottery - a critical methodological finding

A multi-seed audit revealed that single-seed devel selection is unreliable on this small dataset.



(a) Seed audit: devel vs. test across seeds.



(b) Devel→test slope, key configs.

Figure 9: The `mod2` devel-best seed (2345) is a +2.4σ outlier on devel and the *worst* seed on test — devel and test ranks are anti-correlated. The relative-position model (`mod9`), by contrast, generalises positively across all seeds. Its devel-selected seed (777) is the System 2.2 champion.

5.6 Best results and discussion

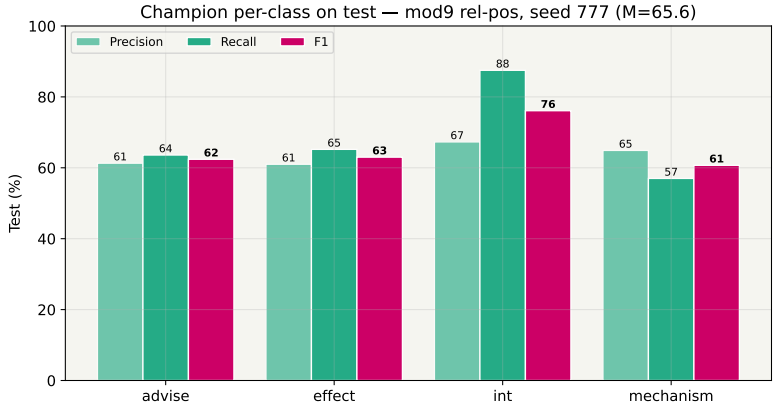


Figure 10: System 2.2 champion (`mod9` rel-pos, seed 777) per-class on test, M-F1 = 65.6. As in 2.1, `int` is the strongest class (F1 = 76.1).

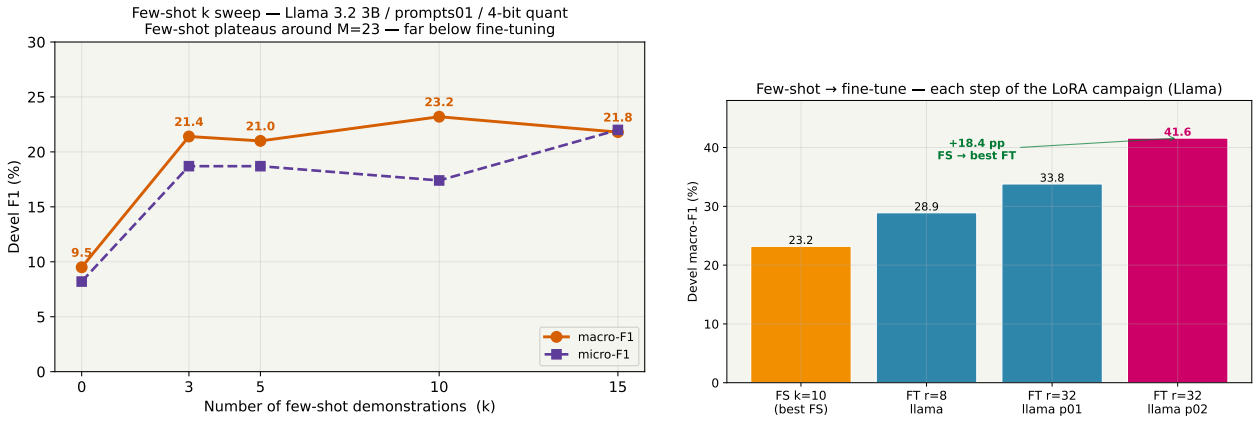
System 2.2 final: BiLSTM + relative-position embedding, devel-selected seed 777 → test M-F1 = 65.6, within 1.2 pp of the ML champion. The relative-position embedding is the single most impactful change (biggest test gain in the campaign), and a clean architectural conclusion emerges: **relative position and the BiLSTM are complementary, not interchangeable** — a pure CNN with rel-pos but no BiLSTM collapses to 0.0. The recurring lesson is the same as 2.1: *input representation* > *architecture* > *hyperparameters*, and single-seed peaks must be treated with suspicion.

6 System 2.3 - DDI with Large Language Models

6.1 Starting point and output format

The LLM system frames DDI as **single-label generation**: the two target drugs are masked in the sentence as `[DRUG1]` / `[DRUG2]` (other entities as `[DRUG_OTHER]`), and the model is asked to emit exactly one of the five class labels. We work with two 4-bit-quantised 3B instruction models, **Llama-3.2-3B** and **Qwen-2.5-3B**, on the Boada cluster. The brief asks us to experiment along three axes: prompt adaptation, few-shot example selection, and fine-tuning (examples + hyperparameters).

6.2 Few-shot experiments



(a) Few-shot k-sweep (Llama, `prompts01`). (b) Few-shot → fine-tune jump.

Figure 11: Few-shot inference plateaus around macro-F1 = 23 regardless of the number of demonstrations k — far below even the weakest fine-tune. Fine-tuning is essential: each step of the LoRA campaign adds clear macro-F1.

6.3 Fine-tuning experiments

We fine-tuned both models with LoRA at two ranks (r=8, r=32, α=2r) and two prompt templates. `prompts02` fixes a typo in the `advise` class definition and strengthens the "do not invent an interaction" instruction for `null`.

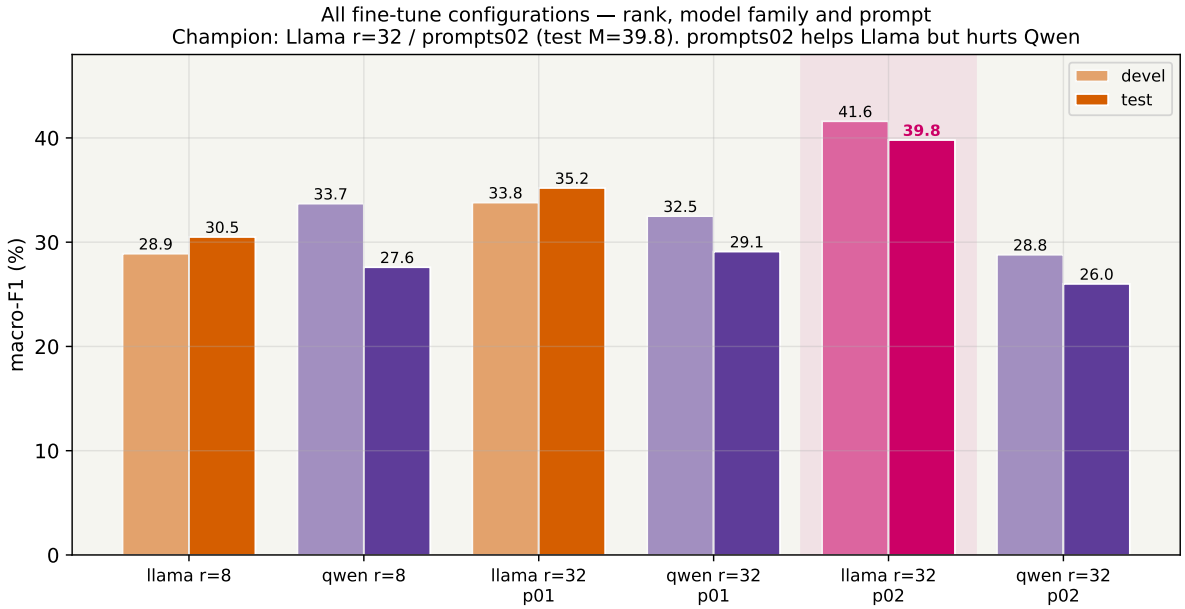


Figure 12: All fine-tune configurations (devel + test macro-F1). The champion is **Llama** r=32 / **`prompts02` (test M-F1 = 39.8)**. Doubling the LoRA rank (8 → 32) and improving the prompt each add ≈+5 pp on Llama — prompt quality is as powerful a lever as adapter capacity.

6.4 Prompt brittleness across model families - a key finding

The most interesting result of the campaign is that the *same* prompt improvement has *opposite* effects on the two model families.

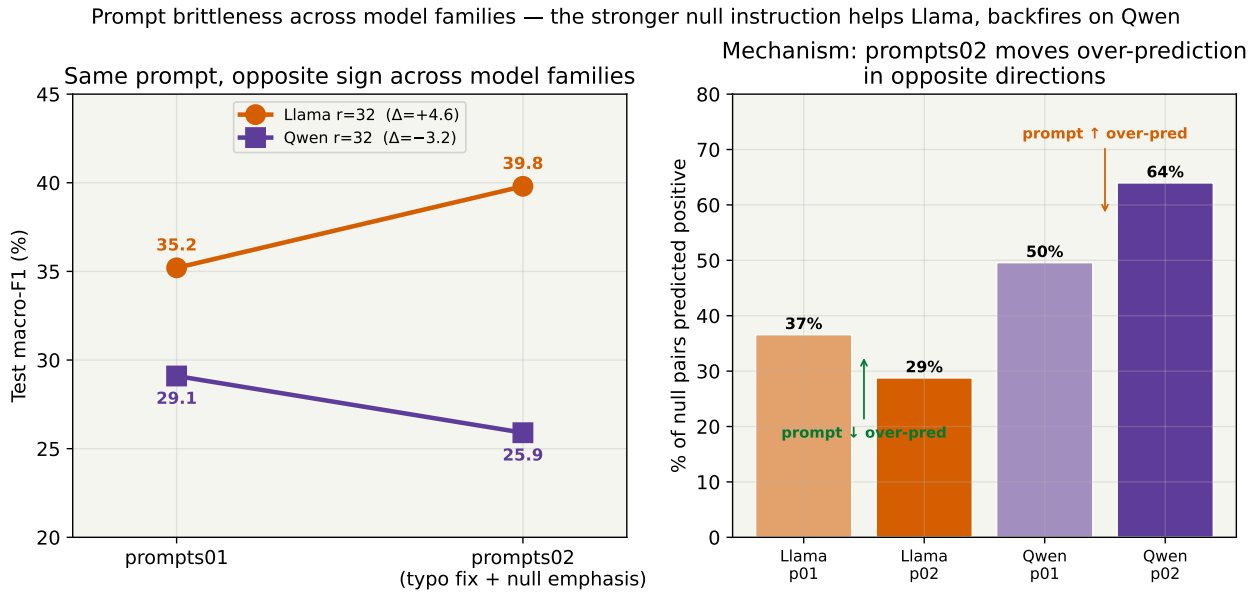
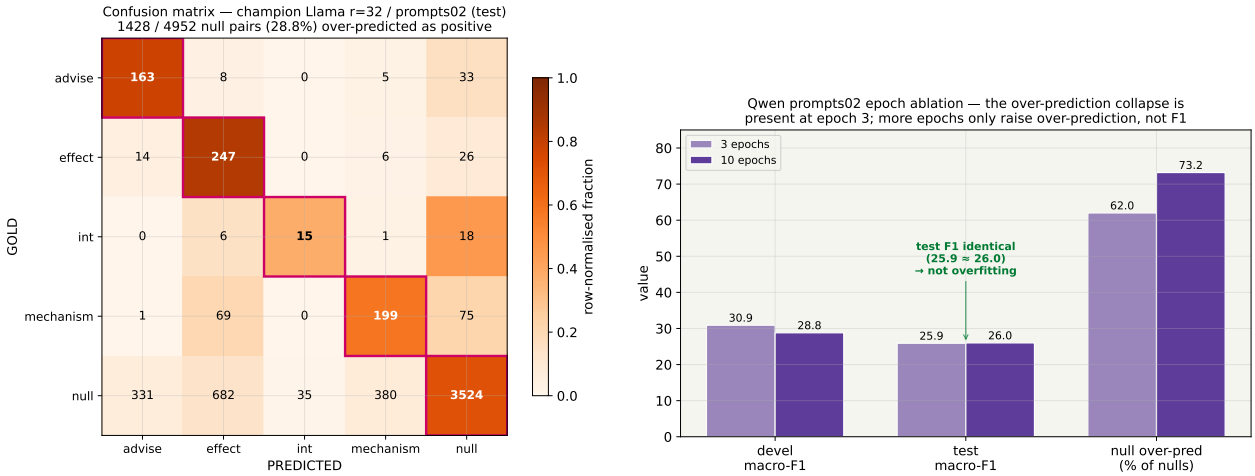


Figure 13: `prompts02` helps Llama (+4.6 pp test) but *hurts* Qwen (−3.2 pp). The mechanism (right) is the null over-prediction rate: the stronger `null` instruction made Llama predict positives *less* (36.6 %→28.8 %) but made Qwen predict them *more* (49.6 %→64 %). The same instruction is read as caution by one model and as licence by the other.

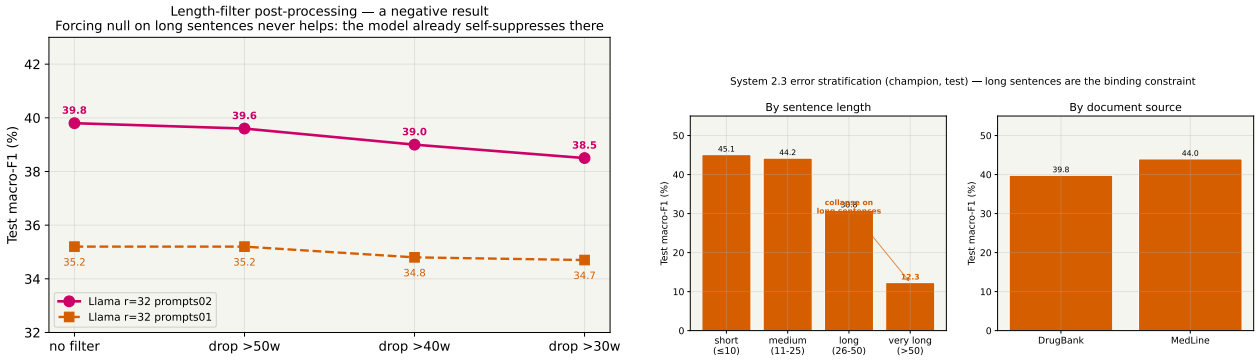


(a) Champion confusion matrix (test).

(b) Qwen epoch ablation.

Figure 14: Left: even the champion over-predicts — 28.8 % of all `null` pairs receive a positive label (precision ≪ recall on every class). Right: a 3-epoch vs. 10-epoch ablation on Qwen shows the over-prediction collapse is *not* overfitting — test F1 is identical (25.9 vs. 26.0); more epochs only inflate the over-prediction rate.

6.5 A negative result and error stratification



(a) Length-filter post-processing (negative). (b) Error stratification (champion, test).

Figure 15: Left: forcing `null` on long sentences never helps — the model already self-suppresses there (only ≈2 % of its positive predictions fall in >50-word sentences), so the collapse is a *recall* problem, not a precision one a filter could fix. Right: like 2.1 and 2.2, the LLM degrades sharply on long sentences (M-F1 = 12.3 on >50 words).

System 2.3 final: LoRA fine-tuned Llama-3.2-3B, r=32, `prompts02` → test M-F1 = 39.8. Three transferable findings: (1) fine-tuning is mandatory — few-shot plateaus 17 pp lower; (2) prompt quality is a first-class capacity lever, as strong as LoRA rank, but it is **not transferable across model families** (it helped Llama and hurt Qwen); (3) the binding constraint is the positive/`null` boundary — the model over-predicts interactions, and no inference-time guardrail repairs it.

7 Conclusions: Cross-System Comparison

All three systems attack the *same* DDI task and are scored by the *same* evaluator on the *same* test split, so the numbers are directly comparable.

7.1 Headline test-set results

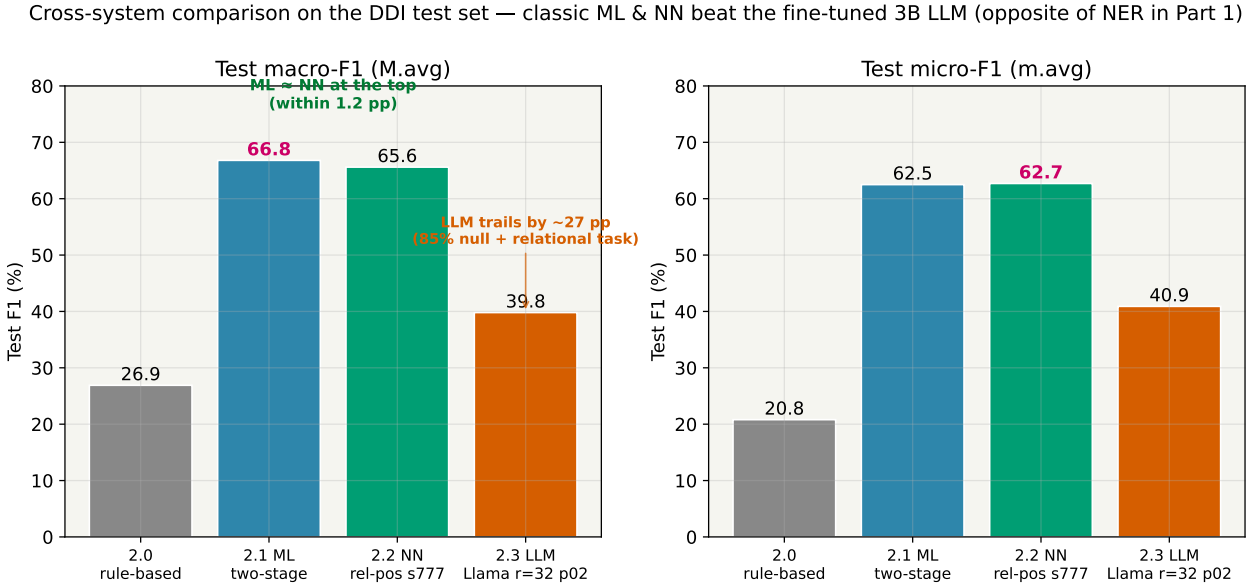
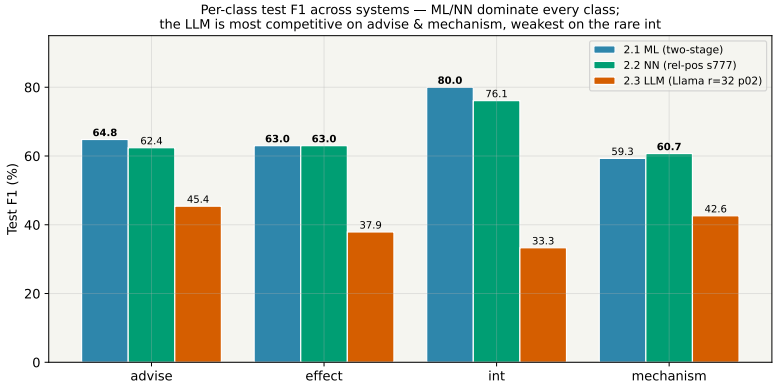


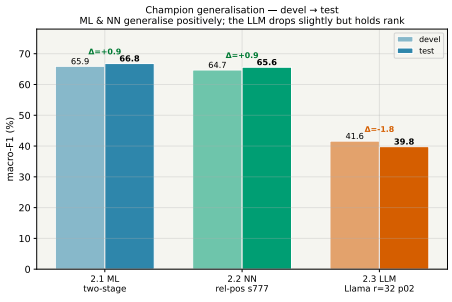
Figure 16: Cross-system test F1. The classical ML system wins macro-F1 (66.8), the neural network is within 1.2 pp (65.6), and the fine-tuned 3B LLM trails both by ≈27 pp (39.8). This is the *mirror image* of Task 1 (NERC), where the fine-tuned LLM won macro-F1 outright.

System (test)	M-F1	m-F1	advise	effect	int	mech.
2.0 rule-based	26.9	20.8	20.1	25.2	50.0	12.3
2.1 ML (two-stage)	**66.8**	62.5	64.8	63.0	**80.0**	59.3
2.2 NN (rel-pos, seed 777)	65.6	**62.7**	62.4	63.0	76.1	**60.7**
2.3 LLM (Llama r=32 p02)	39.8	40.9	45.4	37.9	33.3	42.6

Table 5: Headline test-set comparison (% F1; best positive-class values in bold). ML and NN are statistically a tie at the top; the LLM trails on every class and is weakest on the rare `int`.



(a) Per-class test F1 across systems.



(b) Champion devel→test generalisation.

Figure 17: ML and NN dominate every class; the LLM is most competitive on `advise` and `mechanism` and weakest on the rare `int`. All three champions generalise sensibly from devel to test.

7.2 Trade-offs: effort, cost, explainability, controllability, performance

The brief asks explicitly for a comparison across these five dimensions.

- **Development effort.** 2.1 required the most *thinking* (feature engineering: five mods, a solver/penalty

sweep, the two-stage decomposition) but each idea is a few lines and a 3-minute re-run. 2.2 was moderate (wiring four input channels, the relative-position embedding, architecture flags). 2.3 was the *least code* (env-var knobs for LoRA rank and epochs, a second prompt file) but the longest *wall-clock* per idea by far.

- **Computational cost.** 2.1 runs entirely on CPU: feature extraction ≈3 min, training in seconds — the whole campaign is minutes. 2.2 trains in ≈5–15 min per seed on one GPU (≈1 h for a 5-seed audit). 2.3 is **two orders of magnitude** more expensive: a single fine-tune is 1.5–5.4 GPU-hours and inference alone runs up to ≈5 GPU-hours for Qwen; the full LLM campaign consumed tens of GPU-hours and repeatedly hit the cluster's disk quota.
- **Explainability.** 2.1 is the most interpretable: the learned feature weights are directly inspectable (`same-drug` is the model's strongest signal; class-appropriate trigger verbs follow), and every mod's effect has a concrete cause. 2.2 is intermediate (input-channel ablations are interpretable; hidden states are not). 2.3 is the least: we can *observe* that Qwen over-predicts under `prompts02` but cannot point to why inside a 3B network.
- **Controllability.** 2.3 offers the most direct behavioural handles at inference (a one-line prompt swap visibly reshapes per-class recall) — but those handles are **brittle and non-transferable**, as the opposite Llama/Qwen prompt response shows. 2.1 is controllable through its feature set and stage-1 threshold (a smooth, predictable precision/recall dial); 2.2 only through architecture flags fixed at training time.
- **Performance.** On this task the ordering is unambiguous: **2.1 ML (66.8) ≈ 2.2 NN (65.6) ≫ 2.3 LLM (39.8)** on macro-F1. The classical pipeline wins.

7.3 Why the LLM loses on DDI but won on NERC

The same 3B LoRA recipe *won* macro-F1 on Task 1 (NERC) yet trails by ≈27 pp here. Three structural reasons explain the reversal: **(1) the task is relational**, not a surface property of one span — the answer depends on the syntactic relationship between two marked drugs, which 2.1 encodes explicitly via dependency-path features and the LLM must reconstruct from raw text; **(2) the 85 % `null` imbalance** drives the LLM into systematic over-prediction (it labels ≈29–64 % of true `null` pairs as positive), the single largest error source; **(3) the output is a single token**, so the LLM gets none of the structured-generation advantage that helped it on the span-tagging NERC task. None of these is fixable by prompting at the 3B/4-bit scale our hardware allows.

7.4 When is each system the right choice?

- **Use 2.1 (ML)** as the default for DDI: best macro-F1, CPU-only, fully interpretable, minutes per iteration. On a relational, syntax-driven task with hand-crafted dependency features, classical ML is hard to beat.
- **Use 2.2 (NN)** when a GPU is available and one prefers learned representations to feature engineering; it reaches within 1.2 pp of ML without any hand-crafted syntactic features, and the relative-position embedding is the key to getting there.
- **Use 2.3 (LLM)** only where its flexibility (zero feature engineering, natural-language control) outweighs a large accuracy and cost penalty. At 3B/4-bit it is not competitive for DDI; a larger, full-precision model might narrow the gap but would not change the ranking on this hardware.

Overall conclusion. For DDI-2013 the three paradigms finish **2.1 ML = 66.8 %**, **2.2 NN = 65.6 %**, **2.3 LLM = 39.8 %** macro-F1. The central, task-independent lesson — consistent with Task 1 — is that **representation matters more than architecture**: in 2.1 a feature *combination* plus the two-stage decomposition delivered the gains, in 2.2 the input channels and the relative-position embedding did, and in 2.3 the LoRA rank and the prompt did. But the headline lesson is the *contrast* with NERC: a small fine-tuned LLM can match dedicated systems on a span-extraction task, yet on a *relational* task with heavy class imbalance and syntactic signal, the classical feature-based pipeline remains clearly ahead.